PROJECT DOCUMENTATION

StockWise – Inventory & Stock Management System

Student name: Jasmina Shaikh

AF code: AF05319826

Project Type: Web Application

Technology Stack: PHP, Laravel, MySQL

Development Environment: PHP-compatible local/server environment

Project Level: Beginner to Intermediate

Document Version: 1.0

1. Project Overview

1.1 Project Title

StockWise – Inventory & Stock Management System

1.2 Introduction

StockWise is a web-based application designed to help businesses manage products, stock quantities, product categories, suppliers, and sales-related inventory changes. It provides a centralized way to maintain inventory records and makes it easier to identify products that need replenishment.

The system is planned using PHP with the Laravel framework and MySQL as the database. When a sale is recorded, the corresponding product quantity can be reduced automatically. Low-stock alerts help users identify items that have reached or fallen below their defined reorder level.

1.3 Problem Statement

Maintaining inventory manually in registers or spreadsheets can cause incorrect stock counts, duplicate records, delayed updates, difficulty tracking suppliers, and missed opportunities to reorder products. StockWise aims to organize inventory information in one system and improve the accuracy and visibility of stock records.

1.4 Objectives

- Maintain product and stock information in a centralized database.
- Provide Create, Read, Update, and Delete (CRUD) operations for products.
- Organize products by category.
- Maintain supplier details for inventory purchasing and replenishment.
- Update stock quantities when supported sales or stock transactions are recorded.
- Identify products at or below their low-stock threshold.
- Provide authentication so only authorized users can manage inventory.
- Improve data accuracy and reduce manual inventory work.

1.5 Project Scope

The initial version is intended for small businesses, shops, and other organizations that need basic inventory tracking. Its core scope includes authentication, dashboard and inventory overview, product management, category management, supplier management, and database configuration. Advanced features such as barcode scanning, purchase-order workflows, multi-branch stock, accounting integration, and detailed analytics may be considered for future versions.

2. Functional Requirements

Module	Description
User Authentication	Allows an authorized user to log in and log out.
Dashboard and Inventory Overview	Shows a summary of products, stock levels, and items requiring restocking.
Product Management	Adds, views, edits, and deletes product records.
Product Category Management	Creates and maintains categories used to group products.
Supplier Management	Stores and maintains supplier contact and business details.
Stock Updates	Maintains stock quantities when supported stock-in, stock-out, or sales operations occur.
Low-Stock Alerts	Flags products whose available quantity is at or below the configured threshold.
Database Design and Configuration	Defines tables, relationships, constraints, and Laravel database settings.

2.1 Non-Functional Requirements

- Security: Use authentication, authorization, validation, CSRF protection, and parameterized database access provided by Laravel.
- Performance: Product lists and dashboard summaries should load in a reasonable time.
- Usability: Screens and workflows should be clear and consistent.
- Maintainability: Keep code organized using Laravel routes, controllers, models, migrations, and views.
- Reliability: Validate stock changes and handle database errors appropriately.
- Data Integrity: Prevent invalid quantities and maintain valid relationships between products, categories, and suppliers.

3. Technology Stack

Technology	Purpose
PHP	Server-side programming and business logic.
Laravel	Application framework for routing, controllers, models, validation, middleware, and database operations.
MySQL	Stores user, product, category, supplier, and inventory information.
Eloquent ORM	Maps database tables to Laravel models and supports database queries.
Laravel Migrations	Defines and versions the database structure.
Composer	Manages PHP dependencies.
Laravel Authentication	Provides the foundation for user login and session handling; the exact starter package depends on implementation.
Development Environment	A PHP-compatible local environment and a code editor such as Visual Studio Code.

Note: The project description specifies PHP, Laravel, and MySQL. A particular frontend library or UI framework has not been specified and should only be documented if it is actually used in the implementation.

4. System Architecture

StockWise follows a layered Laravel application structure.

Layer	Responsibility
Presentation Layer	Displays pages and forms to the user through the selected Laravel view/frontend implementation.
Application Layer	Laravel routes, controllers, validation rules, middleware, and business logic process user requests.
Model / Data Access Layer	Eloquent models and query operations interact with the database.
Database Layer	MySQL stores users, products, categories, suppliers, and inventory-related records.

4.1 Request Flow

1. The user opens StockWise and authenticates.
2. The user selects an inventory function, such as viewing or adding a product.
3. Laravel routes the request to the relevant controller.
4. The controller validates the input and applies authorization checks.
5. The model or query layer reads or updates the MySQL database.
6. Laravel returns a response showing the result or a validation/error message.

4.2 Stock Update Flow

7. An authorized user records a supported stock or sales operation.
8. The application checks that the product exists and that the requested quantity is valid.
9. For a sale or stock-out operation, the application verifies sufficient available stock.
10. The application updates the stock quantity and records the operation if transaction history is implemented.
11. The dashboard and low-stock status reflect the updated quantity.

For consistency, stock-changing operations should use database transactions when multiple related database writes must succeed together.

5. Database Design

5.1 Proposed Database Name

stockwise

The following is a suggested schema for the described modules. Column names and relationships should be aligned with the final implementation before submission.

5.2 Table: users

Column	Data Type	Description
id	BIGINT, Primary Key	Unique user identifier.
name	VARCHAR(255)	User display name.
email	VARCHAR(255), Unique	Login email address.
password	VARCHAR(255)	Hashed password; never store plain-text passwords.
created_at / updated_at	TIMESTAMP	Laravel-managed timestamps.

5.3 Table: categories

Column	Data Type	Description
id	BIGINT, Primary Key	Unique category identifier.
name	VARCHAR(150)	Category name.
description	TEXT, Nullable	Optional category description.
created_at / updated_at	TIMESTAMP	Record timestamps.

5.4 Table: suppliers

Column	Data Type	Description
id	BIGINT, Primary Key	Unique supplier identifier.
name	VARCHAR(150)	Supplier or business name.
contact_person	VARCHAR(150), Nullable	Supplier contact person.
email	VARCHAR(255), Nullable	Supplier email address.
phone	VARCHAR(30), Nullable	Contact number.
address	TEXT, Nullable	Supplier address.

created_at / updated_at	TIMESTAMP	Record timestamps.

5.5 Table: products

Column	Data Type	Description
id	BIGINT, Primary Key	Unique product identifier.
name	VARCHAR(200)	Product name.
sku	VARCHAR(100), Unique if used	Stock keeping code or product code.
category_id	BIGINT, Foreign Key	References categories.id.
supplier_id	BIGINT, Foreign Key, Nullable	References suppliers.id where a primary supplier is recorded.
quantity	INT UNSIGNED	Current available stock quantity.
reorder_level	INT UNSIGNED	Threshold used for low-stock alerts.
unit_price	DECIMAL(10,2), Optional	Selling or reference price, if tracked.
created_at / updated_at	TIMESTAMP	Record timestamps.

5.6 Optional Table: stock_transactions

Column	Data Type	Description
id	BIGINT, Primary Key	Unique transaction identifier.
product_id	BIGINT, Foreign Key	References products.id.
type	VARCHAR(30)	For example, stock_in, stock_out, or sale.
quantity	INT UNSIGNED	Quantity added or removed.
notes	TEXT, Nullable	Optional explanation or reference.
created_at	TIMESTAMP	Transaction date and time.

5.7 Database Relationships

- One category can contain many products.
- A supplier can be associated with multiple products when the application uses a primary supplier per product.
- A product can have multiple stock transaction records if transaction history is implemented.
- Users are authenticated before accessing protected inventory functions.

5.8 Integrity and Constraints

- Use primary keys for unique row identification.
- Use foreign keys for category, supplier, and stock-transaction relationships.
- Validate that quantity and reorder_level are not negative.
- Use a unique constraint for SKU or email where applicable.
- Use database transactions for operations that update stock and create transaction history together.

6. Module Description

6.1 User Authentication (Login and Logout)

The user provides login credentials. The application verifies the credentials and establishes an authenticated session. Protected inventory routes should be accessible only to authenticated users. Logout ends the session.

- Store passwords using secure password hashing.
- Use Laravel authentication and middleware or the chosen equivalent.
- Validate credentials and protect routes from unauthenticated access.
- Do not expose passwords or sensitive configuration in error messages.

6.2 Dashboard and Inventory Overview

The dashboard presents a summary of inventory information. Depending on the implemented features, it can show the total number of products, category and supplier counts, stock quantities, and products that have reached their reorder level.

6.3 Product Management

Authorized users can add a product, view product details and lists, update product information, and delete a product where permitted. Required fields and references to categories or suppliers must be validated before saving.

6.4 Product Category Management

This module allows users to create, view, update, and delete product categories. The application should handle categories already linked to products according to its chosen deletion policy, such as preventing deletion or requiring reassignment.

6.5 Supplier Management

This module maintains supplier details, including business name and optional contact information. Supplier records can be created, viewed, updated, and deleted subject to product relationship rules.

6.6 Stock Management and Low-Stock Alerts

The stock quantity should be updated through valid inventory operations. When a sale is recorded, the sold quantity should be deducted from the relevant product. The system should prevent a transaction from reducing stock below zero unless the business rules explicitly allow it. A product should be flagged as low stock when its quantity is less than or equal to its reorder level.

6.7 Database Design and Configuration

Laravel database credentials and connection settings are configured through environment variables. Migrations define the database schema, and Eloquent models represent the main entities. Sensitive environment files should not be committed to a public repository.

7. Suggested Laravel Project Structure

```
stockwise/
├── app/
│   ├── Models/
│   │   ├── Product.php
│   │   ├── Category.php
│   │   ├── Supplier.php
│   │   └── StockTransaction.php  (optional)
│   └── Http/Controllers/
│       ├── ProductController.php
│       ├── CategoryController.php
│       ├── SupplierController.php
│       └── DashboardController.php
├── database/
│   └── migrations/
├── resources/
│   └── views/
├── routes/
│   └── web.php
├── config/
├── .env                (local secrets; do not commit)
├── composer.json
└── README.md
```

This is a suggested structure. Actual filenames and folders may differ depending on the Laravel version, authentication starter kit, and project implementation.

8. Development Methodology

12. Requirement Analysis: Confirm the inventory tasks, users, and business rules.
13. System Design: Plan the modules, database schema, relationships, and request flow.
14. Project Setup: Create the Laravel project and configure the environment.
15. Database Development: Create migrations and models for products, categories, suppliers, and users.
16. Module Development: Implement authentication, dashboard, and CRUD operations.
17. Inventory Logic: Implement stock changes and low-stock checks.
18. Validation and Security: Apply validation, authorization, CSRF protection, and safe database access.
19. Testing: Test normal, invalid, and boundary scenarios.
20. Deployment: Configure environment variables and deploy to a suitable PHP/MySQL server.

9. Validation and Exception Handling

The application should validate all user-controlled data on the server before storing or using it.

- Product name must not be empty.
- Quantity and reorder level must be valid non-negative integers.
- Category and supplier identifiers must reference existing records when provided.
- Price must be numeric and non-negative if price is part of the implementation.
- SKU must be unique if the application uses SKU tracking.
- A sale or stock-out operation must not exceed available stock.
- Invalid record identifiers must be handled without exposing internal errors.
- Database exceptions should be logged securely and user-facing messages should remain understandable.

Laravel validation, Eloquent, authorization middleware, and database transactions can be used to implement these checks. The final documentation should reflect the actual rules and error-handling behavior in the code.

10. Testing

10.1 Test Cases

Test ID	Test Scenario	Expected Result
TC01	Log in with valid credentials	Authorized dashboard opens.
TC02	Log in with invalid credentials	Login is rejected with a suitable message.
TC03	Add a product with valid details	Product is saved in the database.
TC04	Submit a product with missing required data	Validation errors are shown and invalid data is not saved.
TC05	Edit an existing product	Updated product details are saved.
TC06	Delete a product that can be deleted	Product is deleted or handled according to relationship rules.
TC07	Create or update a category	Category data is saved correctly.
TC08	Create or update a supplier	Supplier data is saved correctly.
TC09	Record a sale with sufficient stock	Stock quantity is reduced correctly.
TC10	Attempt a sale exceeding available stock	Operation is rejected and stock remains unchanged.
TC11	Set quantity at or below reorder level	Product is flagged as low stock.
TC12	Open a protected route while logged out	Access is denied or redirected to login.
TC13	Database connection fails	Controlled error handling occurs; sensitive details are not exposed.

10.2 Types of Testing

- Unit Testing: Tests individual validation rules and business functions.
- Integration Testing: Tests Laravel-to-MySQL operations and related stock updates.
- Functional Testing: Verifies that each module meets its requirements.
- Security Testing: Checks authentication, authorization, input validation, and safe database access.
- User Acceptance Testing: Confirms that the workflows suit the intended inventory users.

Record actual test results, screenshots, and defects after executing the application; expected results in this document are planned test outcomes, not claims that testing has already passed.

11. Installation and Execution

11.1 Prerequisites

- PHP version compatible with the selected Laravel version.
- Composer dependency manager.
- MySQL server.
- A local development environment or PHP-compatible web server.
- Code editor such as Visual Studio Code.

11.2 Setup Steps

21. Install PHP, Composer, and MySQL, then verify they are available in the terminal.
22. Create or open the StockWise Laravel project directory.
23. Install project dependencies using Composer if the project has been provided as source code.
24. Create a MySQL database named stockwise, or use the database name configured for the project.
25. Set DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, and DB_PASSWORD in the local .env file.
26. Generate an application key if required by the project setup.
27. Run the project migrations to create the tables.
28. Configure the authentication setup and create an authorized account using the project's supported process.
29. Start the Laravel development server and open the local address shown by the command.
30. Test authentication, product CRUD, categories, suppliers, stock updates, and low-stock alerts.

```
Typical Laravel commands (adjust to the actual project):\ncomposer install\nphp artisan
key:generate\nphp artisan migrate\nphp artisan serve
```

Do not commit the .env file or production credentials. Exact setup commands may vary depending on whether the project is newly created, cloned from a repository, or already configured.

12. Limitations

- The initial scope focuses on core inventory operations.
- Advanced purchase-order and vendor payment workflows are not included unless implemented separately.
- Barcode scanning and automated purchase suggestions are not included in the stated scope.
- Multi-warehouse or multi-branch inventory is not specified.
- Detailed financial accounting and tax calculations are outside the stated scope.
- Reports and analytics may be limited to the information shown on the dashboard.

13. Future Enhancements

31. Stock transaction history with audit details.
32. Purchase orders and supplier delivery tracking.
33. Barcode or QR-code product lookup.
34. Export inventory reports to PDF or Excel.
35. Email notifications for low-stock products.
36. Multi-warehouse or multi-branch inventory support.
37. Sales reports, stock movement charts, and analytics.
38. Role-based permissions for administrators and staff.
39. Product image uploads and SKU/barcode generation.
40. API integration with point-of-sale or e-commerce systems.

14. Conclusion

StockWise is designed to provide a centralized way to manage products, categories, suppliers, and stock quantities. By using PHP, Laravel, and MySQL, the project demonstrates common web-application concepts such as authentication, CRUD operations, relational database design, validation, and structured application development. Stock updates and low-stock alerts can help users maintain more accurate inventory records and identify replenishment needs. The system can be extended with transaction history, reporting, and other features as business requirements grow.

15. References

41. PHP Documentation — https://www.php.net/docs.php
42. Laravel Documentation — https://laravel.com/docs
43. MySQL Documentation — https://dev.mysql.com/doc/
44. Composer Documentation — https://getcomposer.org/doc/
45. OWASP Web Application Security Guidance — https://owasp.org/

Documentation note: This document is a project-specific draft based on the supplied StockWise description and the structure of the provided sample documentation. Update the proposed schema, screenshots, implemented modules, test results, and setup steps to match the actual application before academic or professional submission.